

03 - MongoDB Data Structures

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM



Learning Objectives

- Understand MongoDB Data Modeling
- Understand the different data structures in MongoDB
- Understand the `_id` field
- Perform basic MongoDB CRUD

1

Scenario for the Day



Context Setting

- Rural Savings would like to start their digital transformation with PRIME, their core banking system for retail credit card accounts.
- Meanwhile, Fitness Last would just like a simple app to test the waters of this initiative. The app should be able to show members of the gym how many people are in a particular gym location at a given time.

2

Data Modeling



MongoDB Data Modeling Steps

1. Forget ERD and Relational Databases.
2. List down your access patterns.
3. Design your collections based on your access patterns.



Access Patterns: Rural Savings Mobile Banking

Access Patterns : The list of expected transactions with your database.

1. Login and show all accounts
2. Show account details and credit limits
3. Change customer details
4. Show credit card transactions



Database Relationships

- **One-to-One** – Each instance of Entity A is associated with exactly one instance of Entity B, and vice versa.
- **One-to-Many** – Each instance of Entity A can be associated with one or more instances of Entity B, but each instance of Entity B is associated with only one instance of Entity A.
- **One-to-"Squillions"** – Each instance of Entity A can be associated with an unlimited number of instances of Entity B. This is essentially a One-to-Many relationship, just emphasizing a potentially large number.
- **Many-to-Many** – Each instance of Entity A can be associated with multiple instances of Entity B, and each instance of Entity B can also be associated with multiple instances of Entity A.



Many-to-Many Example

Joint accounts in a bank

- A joint account has many customers
- A customer may have many accounts



To embed or not to embed?

Rule of thumb: Always embed unless you have a compelling reason not to. If a value frequently changes, it's best not to embed.

- **One to Many and Many to Many** - Embed only if you are certain of the max number of records possible
- **One to Many Lookup Tables** - Do not embed (basically repeating data)
- **One to Squillions** - Do not embed

The goal is to:

1. Reduce the number of collections to be queried as much as possible
2. Reduce the number of queries and make use of application caching



Data Duplication

- Embedding introduces some level of data duplication. In our example, accountNumbers are duplicated across customerAccounts and transactions collection.
- Data duplication is not an issue **provided that it serves the purpose of allowing accessing of data together.**



Access Patterns: Rural Savings Mobile Banking

Relationships of our Access Patterns

- Login and show all accounts – One customer can have many accounts (One to Many)
- Show account details and balances – An account can have one credit limit (One to One)
- Change customer details – No relationship
- Transfer Money to Another Account – One account can have squillion transfers (One to Squillion)



Rural Savings Data Model: customerAccounts Collection

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  accounts: [
    <See box on right for contents>
  ]
}
```

```
{
  accountNumber: "RSA09129839",
  creditLimit: 100000.00,
  branch: "Katipunan"
},
{
  accountNumber: "RSA0283782",
  creditLimit: 5000000.00,
  branch: "BGC"
}
```



Rural Savings Data Model: transactions Collection

```
{  
  customerNumber: "RSC981283",  
  accountNumber: "ABC0912039",  
  transactionNumber: "RSTX0001",  
  Type: "debit",  
  Amount: 600.00,  
  Vendor: "Potato Corner",  
  transactionDate: "2025-02-09"  
}
```



Access Patterns: Rural Savings Mobile Banking

Access Patterns : The list of expected transactions with your database.

- ✓ Login and show all accounts: customerAccounts
- ✓ Show account details and creditLimit: customerAccounts
- ✓ Change customer details: customerAccounts
- ✓ Show credit card transactions: transactions



Other Modelling Patterns

- **Inheritance** - entities with a significant number of common fields are stored together in a single collection.
- **Schema Versioning** - schema version field is added to handle schema evolution
- **Computed Values** - Pre computed values are stored especially when they are expected to be accessed.



Inheritance: customerAccounts Collection

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  accounts: [
    <See box on right for contents>
  ]
}
```

```
{
  accountNumber: "RSA09129839",
  creditLimit: 100000.00,
  accountType: "credit",
  branch: "Katipunan"
},
{
  accountNumber: "RSA0283782",
  accountType: "savings",
  balance: 6200,
  branch: "BGC"
}
```



Use Cases of Inheritance

- If similar objects/documents are allowed to be stored in the same collection and the access pattern requires it.
 - ▷ In our example, bank data usually is not stored in the same collection unless there has been a pre-aggregation process that aggregates all data, thus inheritance is not applicable.
 - ▷ Savings/Current Accounts and Credit Card Accounts stream from different sources
- Performance bottlenecks can be solved by using inheritance.



Schema Versioning: customerAccounts Collection

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  schemaVersion: 1.0
}
```

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  customerMobileNumber: "9173629812",
  schemaVersion: 2.0
}
```



Use Cases of Schema Versioning

- Handles schema evolution without full migration unlike RDBMS
- Querying documents of a specific version
- Handling code level errors that may occur due to the misalignment of available attributes.
- Human readable change management identifier for collections



Computed Values: customerAccounts Collection

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  accounts: [
    <See box on right for contents>
  ]
}
```

```
{
  accountNumber: "RSA09129839",
  creditLimit: 100000.00,
  balance: 25000.00,
  branch: "Katipunan"
},
{
  accountNumber: "RSA0283782",
  creditLimit: 5000000.00,
  balance: 6200,
  branch: "BGC"
}
```



Use Cases of Computed Values

- When a non-real time numeric value is accepted by the client system (e.g. Credit Card Balance updated after daily bank clearing, telco plan bill)



Access Patterns: Rural Savings Mobile Banking

Relationships of our Access Patterns

- Login and show all accounts – One customer can have many accounts (One to Many)
- Show account details and balances – An account can have one credit limit (One to One)
- Change customer details – No relationship
- Transfer Money to Another Account – One account can have squillion transfers (One to Squillion)



Rural Savings Data Model: customerAccounts Collection

```
{
  customerName: "Luka Doncic",
  customerNumber: "RSC981283",
  customerAddress: {
    city: "Los Angeles",
    country: "United States"
  },
  accounts: [
    <See box on right for contents>
  ]
  schemaVersion: 1.0
}
```

```
{
  accountNumber: "RSA09129839",
  creditLimit: 100000.00,
  balance: 100.00,
  branch: "Katipunan"
},
{
  accountNumber: "RSA0283782",
  creditLimit: 5000000.00,
  Balance: 20.00
  branch: "BGC"
  schemaVersion: 1.0
}
```



Justification for the Data Model

- Data that will be accessed together are bundled together.
 - Accounts will always be accessed together with the customer.
- Inheritance is usually not applicable in banking systems due to isolation of sources and
- Schema versioning may be applied depending on the use case.
- Computed values such as balance of credit card account is added for ease of access.



Try It Yourself

Access Patterns: Fitness Last Mobile App

- Access account details in the profiles page.
 - ▷ Account attributes: fullName, age, sex, height, weight
- Access enrolled branches in the profiles page. No dedicated branch page needed, just a horizontal box inside the profile showing the branch attributes.
 - ▷ Branch attributes: branchName, address, numberOfMembersInGym
- Access number of members currently in the gym in enrolled branches as a small indicator in the profiles page.
- Access fitness class schedules in a different page in the app.
 - ▷ Class attributes: branchName, className, classStartDateTime, instructor



Try It Yourself

Instructions:

- Work with your group to complete this exercise.
- The expected output are 2 things: the schema/data model, and the justification.
- Put these 2 deliverables together in a single txt document and upload them to the provided Canvas assignment.

Grading:

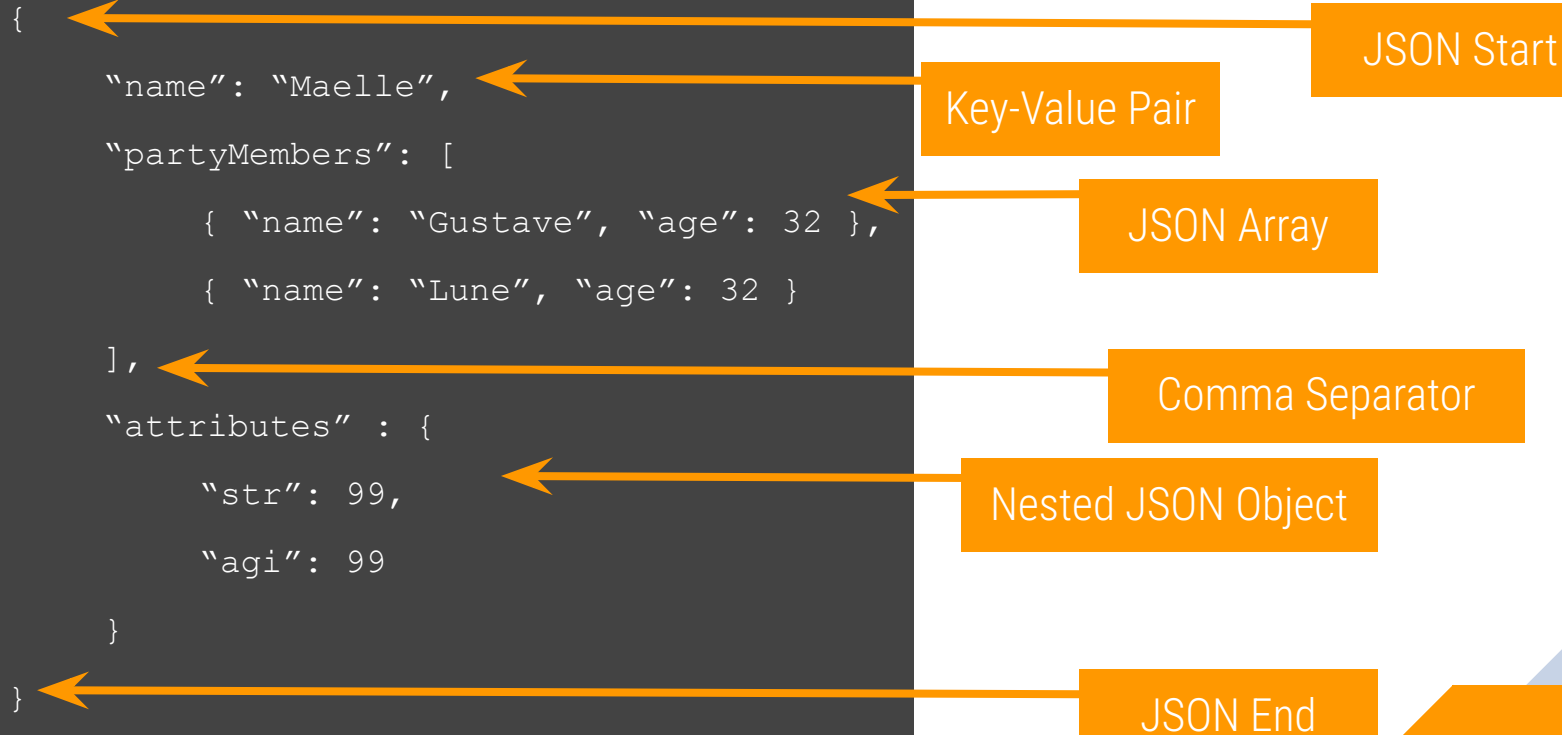
- Completeness of Entities - 10 points
- Correctness of schema - 10 points
- Justification - 10 points

3

JSON 101



JSON Object Structure



4

MongoSh CRUD



Interfacing with MongoDB

There are 3 ways to access MongoDB:

- **mongosh (Mongo Shell)** – a direct CLI that allows CRUD access to MongoDB Databases.
 - ▷ Connecting to mongosh is possible through these ways:
 - ▷ SSH to your Application server and use the mongosh client there. (Recommended)
 - ▷ SSH to your MongoDB server and use the mongosh client there.
 - ▷ Use MongoDB Compass to connect to your Application Server and MongoDB Server, then use the mongosh client there.
 - ▷ **In this lecture, we will be using this. You are free to use any of the setups identified above.**



Interfacing with MongoDB

- **MongoDB Compass** – a user interface developed by MongoDB that allows users to interact with their database.
 - ▷ Connect to your MongoDB Server on port 27017 through your Application Server.
- **Development Toolkit (Like pyMongo)** – a development library that allows CRUD access to MongoDB using functions in a chosen programming language.
 - ▷ Connect to your MongoDB Server on port 27017 either on your Application Server or MongoDB server, wherever there is an installation of Python and pyMongo.



CREATE Syntax

```
db.<collectionName>.insertOne({ <your document here> })  
db.<collectionName>.insertMany([  
<your document array here>  
])
```

`insertOne` accepts a single JSON document while `insertMany` accepts an array of JSON objects. That is why the parameter in `insertOne` is enclosed in `{ }` while the parameter in `insertMany` is enclosed in `[ ]`.



CREATE Syntax: Try It

Try creating a customerAccounts collection in the ruralSavingsNew database and insert the following attributes with fabricated values:

- customerNumber - string
- customerName - string
- age - numeric
- address - object
 - ▷ city - string
 - ▷ country - string
- accounts - array
 - ▷ accountNumber - string
 - ▷ cardNumber - string
 - ▷ branch - string
 - ▷ creditLimit - numeric
 - ▷ activeIndicator - boolean



The _id Field

When you insert a document in a collection, an “_id” field is created. This is created with the type ObjectID.

This is automatically assigned if you have not added it in your insert statement.

You can also make use of one of your existing id fields as _id. This way, your records will become unique according to that field. (NOTE: When you insert using a duplicate id, the operation will fail.)



Basic Data Types

- **String** – Used to store text. It is the most commonly used data type in MongoDB and is stored as UTF-8 encoded text.
- **Numeric** – Represents numbers. MongoDB supports different numeric types such as:
 - ▷ int: 32-bit integer
 - ▷ long: 64-bit integer
 - ▷ double: 64-bit floating-point number
 - ▷ decimal128: 128-bit decimal-based floating-point (for high-precision calculations like money)



Other Data Types

- **Boolean** – Stores a value of either true or false. Useful for flags or binary choices.
- **Timestamp** – Stores a 64-bit value indicating a point in time, primarily used internally by MongoDB for operations like replication and write ordering.
- **Array** – Stores multiple values in a single field as a list. Each element in the array can be of any data type, including other arrays or documents.
- **Object (Embedded Document)** – A nested document structure. This allows documents to contain sub-documents (i.e., objects within objects), enabling rich data models.



READ

For us to effectively make use of the READ functions, clone this repo and run the script to generate the test data:

<https://gitlab.com/csci112-h-2526/lecture3-mongodb-data-structures.git>



READ Syntax

```
db.<collectionName>.find({ <parameters> })
```

The `find` function accepts a parameter still in JSON format. But it contains conditions instead of the usual attributes and values you enter when using insert operations. Conditions are enclosed in `{ }`.



Building a Parameter

The most basic form of a parameter is a key-value pair, with the key as an attribute in your document, and the value that you want to filter with.

```
{  
  <field>: <value>,  
  <field>: <value> #Optional  
}
```

NOTE: The query must always comply to the JSON format.



Building a Parameter – Basics

Example, you want to show your customers in `ruralSavings.customerAccounts` whose age is equal to 30.

```
{  
  "age": 30  
}
```

In SQL, this is the equivalent of `SELECT * FROM ruralSavings.customerAccounts WHERE age = 30;`



Building a Parameter – Objects

Example, you want to show your customers in `ruralSavings.customerAccounts` whose age is equal to 30 and lives in Pasig.

```
{  
  "age": 30,  
  "address.city": "Pasig"  
}
```

In SQL, this is the equivalent of `SELECT * FROM ruralSavings.customerAccounts WHERE age = 30 AND cityAddress = "Pasig";`



Building a Parameter – Arrays

Example, you want to show your customers in `ruralSavings.customerAccounts` whose age is equal to 30 and has an active account in any branch.

```
{  
  "age": 30,  
  "accounts.activeIndicator": true  
}
```

NOTE: This will return the whole document that matches the criteria, not the active accounts only. You have to use post processing to do this.



Building a Parameter – Array Alternative

Example, you want to show your customers in `ruralSavings.customerAccounts` whose age is equal to 30 and has an active account in any branch.

```
{  
  "age": 30,  
  "accounts": { $elemMatch: { "activeIndicator": true } }  
}
```

NOTE: This will return the whole document that matches the criteria, not the active accounts only. You have to use post processing to do this.



Basic Operators

Sometimes, parameters need operators for more complex querying. Here are query operators that can be used for non-array and non-nested object attributes:

\$eq – get documents with value equal to the specified value based on a specified key

\$ne – get documents with value not equal to the specified value based on a specified key

\$in – get documents with value equal to any item in a list of values based on a specified key

\$nin – get documents with value not equal to any item in a list of values based on a specified key

\$elemMatch – get documents that match a criteria set to arrays



Basic Operators

Example, you want to show your customers in ruralSavings.customerAccounts whose age is NOT equal to 30 and lives in Pasig and Taguig.

```
{  
  "age": { $ne: 30 },  
  "address.city": { $in: ["Pasig", "Taguig"] }  
}
```

In SQL, this is the equivalent of `SELECT * FROM ruralSavings.customerAccounts WHERE age <> 30 AND cityAddress in ("Pasig", "Taguig");`



Numeric and Timestamp Operators

For numeric and timestamp attributes, you can use the following operators to query documents with attribute values less than or greater than the specified value in the query.

\$lt – get documents with values less than the specified value base on a specified key

\$lte – get documents with value less than or equal to the value based on a specified key

\$gt – get documents with value greater than the specified value based on a specified key

\$gte – get documents with value greater than or equal to the specified value based on a specified key



Numeric and Timestamp Operators

Example, you want to show your customers in ruralSavings.customerAccounts whose age is less than or equal to 30 and lives in Pasig and Taguig.

```
{  
  "age": { $lte: 30 },  
  "address.city": { $in: ["Pasig", "Taguig"] }  
}
```

In SQL, this is the equivalent of `SELECT * FROM ruralSavings.customerAccounts WHERE age <= 30 AND cityAddress in ("Pasig", "Taguig");`



Exists Operator

\$exists – if true, returns all documents that have the specified field. If false, returns all documents that do not have the specified field

TIP: If you don't use \$exists, all documents, regardless of fields, will be returned by the find query.



Exists Operator

Example, you want to show all customers with an age field.

```
{  
  "age": { $exists: true },  
}
```

Note: There is no SQL equivalent for this since SQL is highly structured and requires the exact field names when querying.



Logical Operators

Logical operators are used to add logic to your parameters for more complex querying.

\$or – return documents if at least 1 criterion is met

\$not – return documents if the criteria is not met

\$nor – return documents if neither criterion is met

\$and – return documents if both criteria are met



Logical Operators

What if we want to run this query: `SELECT * FROM ruralSavings.customerAccounts WHERE age <= 30 OR cityAddress in ("Pasig", "Taguig");?`

```
{
  $or: [
    { "age": { $lte: 30 } },
    { "address.city": { $in: ["Pasig", "Taguig"] } }
  ]
}
```

NOTE: Notice how each condition is enclosed in { }.



Nested Logical Operators

You can also nest logical operators for more complex queries:

```
SELECT * FROM ruralSavings.customerAccounts WHERE (age <=
30 OR cityAddress in ("Pasig", "Taguig")) and creditLimit
> 100000;?
```



Nested Logical Operators

```
{
  $and: [
    { $or: [
      { "age": { $lte: 30 } },
      { "address.city": { $in: ["Pasig", "Taguig"] } }
    ] },
    {
      "accounts.balance" : { $gt: 100000 }
    }
  ]
}
```



Querying With Dates

Dates are a different kind of data type in MongoDB. They are not the same as strings. To use dates in your find queries, you must enclose the date parameter into an ISODate() function.

```
{  
  "transactionDate": { $gt: ISODate("2024-01-01") }  
}
```

This is equivalent to:

```
SELECT * FROM ruralSavings.transactions WHERE  
transactionDate > "2024-01-01";
```



Update Syntax

```
db.<collectionName>.updateMany(  
  { <parameters> },  
  { $set : { <update objects>},  
    $rename: { <oldField>, <newField> } }, #this is optional  
  { <options> }  
)
```

The `updateMany` function accepts a parameter still in JSON format, a set of objects to be updated within the `$set` operator, and an optional `$rename` operator. You can also include options that are available to the `updateMany` function.



Update Example

In 2019, Rural Savings was targeted by a massive cybersecurity attack. All of their 2019 data in `ruralSavingsPrime.transactions`, unfortunately, were tampered with. Particularly, 2019 debit transaction vendors were replaced with incorrect values.

Since the data is now fraudulent, the leadership decided to let it go since they didn't have any backups.

Task: For all debit transactions in 2019, replace the value of "type" to "Fraud", add a field "isFraud" with a value of True, and rename the "vendor" field to "fraudVendor".



Update Example

```
{
  $and: [
    { "type": "debit" },
    { "transactionDate": { $gte: ISODate("2019-01-01") } },
    { "transactionDate": { $lt: ISODate("2020-01-01") } },
  ],
},
{
  $set: { "type": "Fraud", "isFraud": true },
  $rename: { "vendor": "fraudVendor" }
}
```



Update Operators

\$unset – removes the specified key from the document

\$inc – increment the value of the specified key, by the specified amount

\$mul – multiply the value of the specified key, by the specified amount

\$rename – rename the specified field

\$push – add a value/key-value to an array inside a document

\$addToSet – add a value/key-value to an array with deduplication (NOTE: If you are adding an object to an array instead of a single value, only values that exactly match the incoming key-value pair will be treated as duplicates.)



UpdateOne vs UpdateMany

The `find` function has a `findOne` and `findMany` version. What happens if you run `updateOne` instead of `updateMany`?

The update will only happen on the first instance the query finds.



Update Options

\$upsert – If true, creates a new document if no documents match the filter. Default is false.

\$writeConcern – Controls the acknowledgment level of the write. Used for data durability and safety.

\$collation – Allows case-insensitive or locale-specific string matching.

\$hint – Specifies the index to use. Useful for performance tuning.



Delete Syntax

```
db.<collectionName>.deleteMany(  
    { <parameters> }  
)
```

When deleting documents in mongoDB, the parameters is similar to that of `findOne/findMany`. When this function is run, MongoDB deletes all documents that match the filter conditions.



Delete Example

The fraud data do not make sense anymore. Delete them from the database.

```
{  
  "fraudTransactions": true,  
}
```



DeleteOne vs DeleteMany

Like the `updateOne` and `updateMany` functions, when you run `deleteOne` instead of `deleteMany`...

Only the first instance the query finds is deleted.

End.

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM